Clickpoint Innovation — Full Platform Architecture & Developer Guide

Complete Reference Manual: Database Schema, Dynamic CMS, Workflows, APIs, and Operations Runbook

Production Grade | Next.js 14 App Router | Supabase PostgreSQL | Prisma ORM 7.9 | pgvector 384-dim | FastAPI Python Microservices | Better Auth | Cloudinary CDN | Version 2.0 (Oct 2026)

Clickpoint Innovation — Full Website & Enterprise Platform Developer Documentation

> **Document Version:** 2.0 (Production Architecture Edition)
> **Status:** Active Production
> **Date of Publication:** October 2026
> **Repository:** `clickpoint-innovation`
> **Author:** Clickpoint Innovation Core Architecture & Engineering Team

Table of Contents

1. Executive Summary & Platform Purpose
2. Technology Stack & Dependency Inventory
   - Core Frameworks & Runtimes
   - Libraries & Packages Analysis (Why Each is Used)
   - Development & Build Tools
3. Complete Database Architecture & Data Modeling
   - Prisma ORM & PostgreSQL Configuration
   - Entity Relationship Diagram (ERD)
   - Authentication & Access Control Tables
   - Singleton Dynamic CMS Page Models
   - Relational Content Entities
   - Careers, Vacancies & Application Engine

- Vector Search & AI Knowledge Models (`pgvector`)

4. Public Website Architecture & User Workflows
   - Route Map & Page Segment Hierarchy
   - Landing Page (12 Dynamic Sections)
   - About Us (`/about`)
   - Services & Dynamic Detail Pages (`/services` & `/services/[id]`)
   - Industries & Sector Solutions (`/industries` & `/industries/[id]`)
   - Case Studies & Portfolio (`/case-studies`)
   - Company Journey & Interactive Milestone Timeline (`/journey`)
   - Technical Blog & Insight Articles (`/blog` & `/blog/[slug]`)
   - Careers, Culture & Job Application Pipeline (`/careers`)
   - Client Testimonials & Public Review Submission (`/testimonials`)
   - Knowledge Accordions & Categorized FAQs (`/faqs`)
   - Contact Us & Quick Enquiry Modal (`/contact`)
   - Custom 404 Error Recovery Page (`/not-found`)
5. Admin Dashboard & Dynamic CMS Engine
   - Dashboard Navigation Architecture & Real-Time Badges
   - Dynamic Page Content Management Workflows
   - Rich Text & Media Publishing (Custom Tiptap Engine)
   - Cloudinary Media Lifecycle (Upload, CDN Delivery & Auto-Purge)
   - Server Actions vs. API Routes Pattern
   - Real-Time Notification System (SSE + BroadcastChannel)
   - Automated Email Dispatch & HR Candidate Tracking System (ATS)
6. AI Chatbot Subsystem (RAG + pgvector + Gemini)
   - Dual-Mode Architecture (FastAPI Microservices vs. Vercel Serverless)
   - Automatic Admin Mutation Reindexing Pipeline
   - Domain Candidate Injection & Multi-Domain Re-Ranking
   - Strict Typography & Zero-Emoji Directives
7. Security, Authentication & RBAC Architecture
   - Better Auth Engine & Session Lifecycle
   - Next.js Route Protection (`middleware.ts`)
   - Brute-Force Rate Limiting & Account Security
   - Service-to-Service Security Header (`X-Internal-Key`)
8. API Route Specifications & Gateway Reference
9. DevOps, Docker Stack & Production Runbook
   - Environment Variables Matrix

- Local Development Commands
- Docker Compose Multi-Container Deployment
- Vercel Production Deployment Workflow
- Database Migration & Seeding Procedures

---

1. Executive Summary & Platform Purpose

Clickpoint Innovation (`clickpoint.com.np`) is an enterprise-grade digital product engineering consultancy headquartered in Kathmandu, Nepal, serving technology partners, enterprises, and high-growth startups globally.

The Clickpoint Innovation web platform is not merely a static marketing site; it is a **fully integrated, headless-capable digital operations ecosystem** featuring:

1. **Dynamic Content Management System (CMS):** Enables non-technical administrators to mutate every headline, statistic, team member profile, service offering, case study, and timeline milestone in real time across the site without code redeployment.
2. **Interactive Client Portals & Lead Capture:** Interactive Quick Enquiry Modals, multi-step contact inquiries, client testimonial submissions with moderation queues, and ATS job applications with PDF resume parsing and automated status emails.
3. **Conversational AI Agent (Clix):** Grounded in live PostgreSQL data using Retrieval-Augmented Generation (RAG), `pgvector` embeddings, semantic re-ranking, and Google Gemini reasoning.
4. **Resilient Dual-Mode Execution:** Capable of running as a microservice mesh on dedicated Linux/Docker servers, while maintaining full fallback functionality when deployed serverless on Vercel.

---

2. Technology Stack & Dependency Inventory

Core Frameworks & Runtimes

- **Next.js 14.2.35 (App Router):** The foundation of the web tier. Utilizes React Server Components (RSC) for zero-client-bundle data fetching, Server Actions for mutations, streaming for progressive rendering, and Route Handlers for RESTful endpoints.
- **React 18.3.1 & React DOM:** Powers concurrent rendering, client state, and custom UI interactions.
- **TypeScript 5.5.4:** Enforces strict compile-time type safety across database schemas, API contracts, and UI components.
- **Node.js 20+ / Python 3.11+:** Node runs the Next.js frontend and admin gateway; Python runs the FastAPI AI ingestion and vector search microservices.

Libraries & Packages Analysis (Why Each is Used)

Package	Version	Layer	Architectural Rationale & Why Used
`prisma` & `@prisma/client`	`^7.9.1`	Database	Next-generation TypeScript ORM. Generates fully type-safe queries, enforces relational constraints, and manages automated PostgreSQL migrations.
`@prisma/adapter-pg` & `pg`	`^7.9.1` / `^8.13.1`	Database	Native PostgreSQL connection adapter allowing Prisma to connect efficiently to Supabase via pooled connections and PgBouncer.
`better-auth` & `@better-auth/prisma-adapter`	`^1.6.25`	Auth	Modern, self-hosted authentication library. Provides cookie-based sessions, password hashing, brute-force rate-limiting, and direct Prisma session storage without external vendor lock-in.
`bcryptjs`	`^3.0.3`	Auth / Security	Cross-platform, zero-dependency password hashing algorithm used for legacy admin credentials and password salting.
`jose`	`^6.2.7`	Security	Universal JSON Web Token (JWT) and cryptographic token utility for stateless edge verification.
`@tiptap/react` & `@tiptap/core`	`^3.31.3`	Admin CMS	Headless, extensible rich text editor built on ProseMirror. Powers the blog and case study WYSIWYG editor.
`@tiptap/starter-kit`	`^3.29.2`	Admin CMS	Bundles essential ProseMirror extensions (bold, italic, headings, lists, blockquotes, code blocks).
`@tiptap/extension-image`	`^3.29.2`	Admin CMS	Embeds responsive images into rich content with Cloudinary CDN integration.
`@tiptap/extension-youtube`	`^3.29.2`	Admin CMS	Enables embedded YouTube video players with custom responsive iframe wrappers.
`@tiptap/extension-link`	`^3.29.2`	Admin CMS	Handles hyperlinking with custom protocols, target behaviors, and sanitation.
`@tiptap/extension-placeholder`	`^3.29.2`	Admin CMS	Renders contextual placeholders when rich text blocks are empty.
`cloudinary`	`^2.10.0`	Media Storage	Official Cloudinary v2 SDK. Handles programmatic image/video uploads, automated WebP/AVIF format optimization, and auto-purging of deleted media assets from the CDN.
`framer-motion`	`^11.3.19`	UI / Animation	Production-ready motion library for layout transitions, modals, accordion collapses, and floating UI animations.

Package	Version	Layer	Architectural Rationale & Why Used
`gsap`	`^3.12.5`	UI / Animation	GreenSock Animation Platform. Powers complex timeline-based scroll triggers, blueprint grid expansions, and SVG path animations.
`nodemailer`	`^9.0.4`	Notifications	Enterprise email transport engine. Dispatches branded HTML transaction emails (applicant confirmations, HR alerts) with embedded CID logo attachments.
`sonner`	`^2.0.7`	UI Notifications	Lightweight, highly customizable toast notification component for optimistic admin feedback and status indicators.
`lucide-react` & `react-icons`	`^0.408.0` / `^5.7.0`	UI Icons	Scalable, accessible SVG icon collections. Lucide is used across admin controls; React Icons provides brand marks (GitHub, LinkedIn, Discord).
`@radix-ui/react-accordion`	`^1.2.0`	UI Primitives	Accessible, unstyled accordion primitive with full keyboard navigation and WAI-ARIA compliance (used in FAQs).
`@radix-ui/react-dropdown-menu`	`^2.1.1`	UI Primitives	Accessible floating menus for user profile actions and table row operations.
`@radix-ui/react-navigation-menu`	`^1.2.0`	UI Primitives	Accessible desktop mega-menu with viewport positioning.
`class-variance-authority` & `clsx`	`^0.7.0` / `^2.1.1`	Styling	Facilitates variant-driven component design (buttons, badges) with declarative props.
`tailwind-merge`	`^2.4.0`	Styling	Efficiently merges Tailwind CSS class lists without style conflicts or order ambiguity (`cn()` utility).
`react-markdown`	`^10.1.0`	Presentation	Renders dynamic markdown into semantic HTML for the AI chatbot bubbles and blog excerpts.

Development & Build Tools

- **Tailwind CSS 3.4.7 & Autoprefixer:** Utility-first CSS engine configured with custom design tokens (blueprint grids, cyan/purple glows, typography).
- **Husky 9.1.7 & lint-staged 17.6.0:** Git pre-commit hooks that run ESLint and type verification on staged files to prevent defective code from entering `main`.
- **ESLint 8.57.1 & eslint-config-next:** Enforces code quality, hooks rules, and accessibility standards.
- **tsx 4.23.1:** Fast TypeScript execution runtime for running database seeds (`prisma/seed.ts`).
- **pnpm 12.4.1:** High-efficiency, content-addressable package manager for deterministic dependency resolution.

---

3. Complete Database Architecture & Data Modeling

Prisma ORM & PostgreSQL Configuration

The application database is hosted on **Supabase AWS ap-southeast-1 (PostgreSQL 15+)** with the `pgvector` extension enabled for AI embeddings.

- **Database Connection Mode:**
  - `DATABASE_URL` : Connects to Supabase PgBouncer Transaction Pooler on port `6543` (`?pgbouncer=true&connection_limit=15`) for high-concurrency serverless executions.
  - `DIRECT_URL` : Connects directly to PostgreSQL on port `5432` for running migrations and schema alters (`prisma db push` / `prisma migrate`).

Entity Relationship Diagram (ERD)

```
erDiagram
    user ||--o{ session : "has many"
    user ||--o{ account : "has many"

    case_study_categories ||--o{ case_studies : "categorizes"
    faq_categories ||--o{ faqs : "categorizes"
    job_categories ||--o{ job_vacancies : "categorizes"
    job_vacancies ||--o{ job_applications : "receives"

    chat_sessions ||--o{ chat_messages : "contains"

    user {
        string id PK
        string name
        string email UK
        boolean emailVerified
        string role
        boolean banned
        datetime createdAt
    }

    session {
        string id PK
        string token UK
        string userId FK
        datetime expiresAt
        string ipAddress
        string userAgent
    }

    job_vacancies {
        string id PK
        string title
        string categoryId FK
        string type
        string location
        string experience
        string salary
        boolean isActive
    }

    job_applications {
        string id PK
        string jobVacancyId FK
        string name
        string email
        string resumeUrl
        string status
        boolean isRead
    }

    case_studies {
        string id PK
        string slug UK
        string title
        string client
        string categoryId FK
```

```
        boolean featured
    }

    blog_posts {
        string id PK
        string slug UK
        string title
        string category
        string author
        boolean featured
    }

    knowledge_chunks {
        string id PK
        string title
        string content
        string sourceType
        string sourceId
        vector embedding
    }
```

Authentication & Access Control Tables

1. `admins` (`model Admin`)

Legacy/direct administrative credentials table:

- `id` (String, PK, CUID)
- `email` (String, Unique)
- `passwordHash` (String)
- `name` (String, Optional)
- `role` (AdminRole Enum: `ADMIN`, `SUPER_ADMIN`)
- `failedLoginAttempts` (Int, default 0): Increments on bad password; triggers account lock.
- `lockedUntil` (DateTime, Optional): Timestamp until which login is rejected.
- `lastLoginAt` (DateTime, Optional)

2. `user` (`model User`)

Better Auth master user record:

- `id` (String, PK)
- `name` (String)
- `email` (String, Unique)
- `emailVerified` (Boolean, default false)
- `image` (String, Optional): Cloudinary avatar URL.
- `role` (String, default "admin")
- `banned` (Boolean, Optional) & `banReason` / `banExpires`
- Relations: `sessions` (1:N), `accounts` (1:N)

3. `session` (`model Session`)

Better Auth session state:

- `id` (String, PK)
- `token` (String, Unique): Encrypted cookie token matching `better-auth.session_token`.
- `userId` (String, FK -> `user.id`, Cascade Delete)
- `expiresAt` (DateTime): Validated by `middleware.ts`.
- `ipAddress` (String, Optional) & `userAgent` (String, Optional)

4. `account` (`model Account`) & `verification` (`model Verification`)

Stores identity provider tokens, hashed passwords, and one-time verification tokens.

Singleton Dynamic CMS Page Models

The platform uses a dedicated **Singleton Document Pattern** for page-level dynamic content. Each table contains a single record with `id = "default"` containing a strongly typed `content` JSON column:

Model	Table Name	Dynamic Content Structure
`LandingPage`	`landing_page`	Contains 12 section configurations: `navbar`, `hero`, `services`, `techStack`, `growth`, `timeline`, `industries`, `caseStudies`, `testimonials`, `faq`, `cta`, `footer`.
`AboutPage`	`about_page`	Contains `aboutHero`, `mission`, `values`, `leadership.team` (array of team members with bio, role, LinkedIn, and expertise), `companyStats`.
`ServicesPage`	`services_page`	Contains `hero`, `categories`, `processSteps` (4-phase agile engineering pods), `techStackMatrix`.
`IndustriesPage`	`industries_page`	Contains sector cards (Fintech, Healthtech, Logistics), compliance certifications, domain capabilities.
`ContactPage`	`contact_page`	Contains `contactInfo` (email, phone, address, office hours), `inquiryOptions`, `quickEnquiryConfig`.
`JourneyPage`	`journey_page`	Contains `eras` (milestone year, title, detailed narrative, gallery photos).
`CareersPage`	`careers_page`	Contains `hero`, `culture`, `benefits` (Remote-first, Learning stipend, Health coverage), `hiringProcess`.
`NotFoundPage`	`not_found_page`	Contains 404 header, subtitle, blueprint graph styling, recommended quick links.
`ChatbotPage`	`chatbot_page`	Contains bot greeting, personality directives, pricing packages, manual node overrides.

Relational Content Entities

1. Case Studies

- `case_study_categories`: Groups portfolio items (e.g. "Fintech", "AI Copilots", "Cloud Microservices").
- `case_studies`:
  - `id` (CUID PK), `slug` (Unique), `title`, `client`, `categoryId` (FK), `category` (String fallback).
  - `description`, `challenge`, `solution`, `results`, `impact`.
  - `metrics` (JSON): Structured metrics (e.g., `[{"label": "Throughput", "value": "10x"}]`).
  - `techStack` (JSON): Array of technology tags.
  - `imageUrl`, `liveUrl`, `buttonText`, `buttonLink`, `featured` (Boolean), `order` (Int).

2. Technical Blog

- `blog_categories`: Unique category names for articles.
- `blog_posts`:
  - `id` (CUID PK), `slug` (Unique), `title`, `excerpt`, `content` (Full HTML from Tiptap).
  - `author`, `authorRole`, `authorAvatar`, `category`, `readTime`.
  - `imageUrl`, `videoUrl` (Optional video embed), `tags` (JSON), `featured` (Boolean).

3. Testimonials & Client Reviews

- `testimonials`:
  - `id` (CUID PK), `clientName`, `clientRole`, `company`, `content`.
  - `rating` (Int, 1 to 5).
  - `avatarUrl` (Cloudinary URL).
  - `isApproved` (Boolean, default false): Controls visibility on the public site.
  - `isRead` (Boolean, default false): Powers admin unread notification badges.
  - `ipAddress`, `userEmail`, `phone`.

4. FAQs

- `faq_categories` 1:N `faqs`:
  - `faqs.question`, `faqs.answer` (Markdown/HTML), `faqs.order`.
  - Relational cascade prevents category deletion if child FAQs exist (`onDelete: Restrict`).

5. Inquiries & Client Leads

- `inquiries` & `contact_inquiries`:
  - Stores incoming form submissions from the Contact Page and Quick Enquiry modal.
  - Fields: `name`, `email`, `phone`, `company`, `service`, `budget`, `message`, `status` (`PENDING`, `IN_PROGRESS`, `CONTACTED`, `COMPLETED`, `ARCHIVED`).
  - `isRead` (Boolean): Powers real-time notification audio chimes and red unread counts.

Careers, Vacancies & Application Engine

- **`job_categories`**: E.g. "Full Stack Engineering", "AI/ML Systems", "Product Design".
- **`job_vacancies`**:
  - `title`, `categoryId` (FK), `type` ("Full-Time", "Internship", "Contract"), `location` ("Remote (Global)", "Kathmandu").
  - `experience` ("Junior", "Mid", "Senior", "Lead").
  - `salary` (e.g. "Rs. 45,000" or "$3,000/mo").
  - `summary`, `responsibilities` (JSON String[]), `requirements` (JSON String[]).
  - `isActive` (Boolean): Controls public visibility and AI chatbot recommendations.
- **`job_applications`**:
  - Links to `jobVacancyId` with `onDelete: Cascade`.
  - Candidate details: `name`, `email`, `phone`, `linkedIn`, `portfolio`, `coverLetter`.
  - Resume storage: `resumeUrl` (Cloudinary secure PDF link), `resumeOriginalName`, `resumeProvider`.
  - `status` (`PENDING`, `REVIEWING`, `SHORTLISTED`, `REJECTED`, `HIRED`).
  - `isRead` (Boolean): Flags new submissions in the admin ATS tab.

Vector Search & AI Knowledge Models (`pgvector`)

- **`knowledge_chunks`** (`model KnowledgeChunk`):
  - `id` (String PK): Deterministic chunk key (e.g. `career-cmse9b4wl0001wex4w21nitan-0`).
  - `title` (String): Indexed title.
  - `content` (Text): Normalized text chunk.
  - `sourceType` (String): Enum identifier (`career`, `about`, `service`, `caseStudy`, `blogPost`, `testimonial`, `faq`, `contact`, `landing`, `journey`, `industry`).
  - `sourceId` (String): Primary key of the originating entity.
  - `url` (String): Deep link back to public website section (e.g. `/about#team`).
  - `embedding` (`vector(384)`): Dense semantic vector generated by `all-MiniLM-L6-v2`.
- **`chat_sessions`** & **`chat_messages`**:
  - Persists conversational history for user sessions with `matchedChunks` JSON metadata.
- **`question_cache`**:
  - Vector-indexed semantic response cache with normalized question hashing.

---

4. Public Website Architecture & User Workflows

Route Map & Page Segment Hierarchy

```
src/app/
├── layout.tsx                # Master HTML layout, ThemeProvider, Toast container, Chatbot widge
├── page.tsx                  # Home Landing Page (12 dynamic sections)
├── not-found.tsx             # Custom 404 Error page with blueprint grid
├── about/page.tsx            # Corporate story, mission, leadership profiles
├── services/
│   ├── page.tsx              # Services catalog & engineering pod methodology
│   └── [id]/page.tsx         # Dynamic service detail pages
├── industries/
│   ├── page.tsx              # Target industries index
│   └── [id]/page.tsx         # Sector-specific compliance & architecture detail
├── case-studies/page.tsx     # Client portfolio, metrics & tech stack tags
├── journey/page.tsx          # Interactive milestone timeline & eras
├── blog/
│   ├── page.tsx              # Technical articles catalog & newsletter capture
│   └── [slug]/page.tsx       # Dynamic blog article page with TOC and author card
├── careers/page.tsx          # Perks, active vacancies list & modal application form
├── testimonials/page.tsx     # Client reviews, star ratings & review submission modal
├── faqs/page.tsx             # Expandable Q&A accordions with instant category filter
├── contact/page.tsx          # Direct contact details, inquiry form & map
└── admin/
    ├── page.tsx              # Login redirect router
    ├── login/page.tsx        # Administrative authentication form
    └── dashboard/page.tsx    # Master CMS & ATS Control Center
```

Landing Page (12 Dynamic Sections)

The landing page (`/`) is built from 12 distinct modular React Server Components reading from `prisma.landingPage`:

1. **Navbar & Announcement Header:** Responsive desktop navigation, mobile drawer, theme toggle, and Quick Enquiry modal trigger.
2. **Hero Section:** Animated blueprint grid background, headline typography, dynamic CTA buttons, and trust metrics.
3. **Services Catalog:** High-level service cards linking to deep service pages.
4. **Tech Stack & Frameworks:** Interactive tool matrix (Next.js, Python, PostgreSQL, AWS, Docker).
5. **Growth & Impact Metrics:** Counter animations showcasing production uptime (99.99%), project completions, and team scaling.
6. **Milestone Timeline:** Condensed company journey preview.
7. **Industries Served:** Sector highlights with compliance badges.
8. **Case Studies Showcase:** High-impact client projects with verifiable metrics (+300% ROI).
9. **Client Testimonials:** Carousel of approved client reviews with star ratings.
10. **FAQ Accordion:** Essential questions regarding engineering pods, pricing, and IP ownership.

11. **Conversion CTA Banner:** Final consultation booking trigger.
12. **Footer:** Quick links, office address, social links, and legal notices.

Careers & ATS Job Application Workflow

1. **Browsing:** Visitor visits `/careers`, reviews remote-first perks, engineering culture, and browses active vacancies filtered from `prisma.jobVacancy` (`where: { isActive: true }`).
2. **Application Trigger:** Clicking "Apply for Position" opens a modal pre-filled with the selected vacancy.
3. **Upload & Submission:**
   - Candidate attaches resume PDF.
   - Endpoint `/api/jobs/apply` uploads the resume to Cloudinary (`folder: clickpoint/resumes`, `resource_type: raw`).
   - Creates a new `JobApplication` record with `status: PENDING` and `isRead: false`.
4. **Real-Time Notification:** The submission triggers an SSE event to the admin dashboard, causing the "Job Applied" badge to increment and sounding an audio alert.
5. **Automated Confirmation:** Nodemailer dispatches an automated branded confirmation email to the applicant.

---

5. Admin Dashboard & Dynamic CMS Engine

Dashboard Navigation Architecture & Real-Time Badges

The administrative portal (`/admin/dashboard`) is organized into five operational groups:

```
[DASHBOARD & ANALYTICS]
  ├── Dashboard Overview (Key metrics, recent leads, quick stats)
  └── Real-time Analytics (Traffic, inquiry volume, conversion rates)

[DYNAMIC CHATBOT APP]
  └── Chatbot Manager (Personality, pricing packages, manual node picker, live tester)

[WEBSITE PAGES (DYNAMIC CONTENT)]
  ├── Landing Page (12 Sections editor)
  ├── About Us Page (Mission, values, leadership team members)
  ├── Services Pages (Catalog & detail templates)
  ├── Industries Pages (Sector cards & compliance)
  ├── Case Studies Page (Portfolio items & metric highlights)
  ├── Company Journey Page (Milestone timeline eras)
  ├── Blog Pages (Article publisher, categories, newsletter)
  ├── Careers Page (Culture, perks & active vacancies)
  ├── Testimonials Page (Review moderation queue [Badge: New])
  ├── FAQs Page (Category manager & Q&A accordions)
  ├── Contact Us Page (Contact details & lead settings)
  └── 404 Error Page (Blueprint grid & fallback links)

[LEADS & INQUIRIES]
  ├── Client Inquiries (Contact messages & quick enquiries [Badge: Unread])
  └── Job Applied (Applicant tracking system [Badge: Unread])

[SYSTEM]
  ├── Security Logs (Authentication audit trail)
  └── System Settings (Sound preferences, platform defaults)
```

Dynamic Page Content Management Workflows

All CMS edits follow an **Optimistic Mutation Pipeline**:

1. **Form Edit:** Admin modifies fields (e.g. updating a team member's role or adding a new service card).
2. **Server Action Dispatch:** Calls the corresponding server action (e.g., `saveAboutUs(formData)`).
3. **Database Write:** Prisma executes an atomic update on the singleton record or relational model.
4. **Cache Invalidation:** Calls `revalidatePath("/about")` and `revalidatePath("/")` to purge Next.js static page cache.
5. **AI Vector Resync:** Invokes `triggerReindexBackground()` to re-embed modified content into `knowledge_chunks`.
6. **Toast Feedback:** Sonner displays an instant success toast; dashboard navigation reflects updated fields.

Rich Text & Media Publishing (Custom Tiptap Engine)

Blog posts and case study narratives are authored using an advanced custom Tiptap rich text editor (`src/components/admin/rich-text-editor.tsx`):

- **Custom HTML5 Native Video Node (`VideoExtension`):** Supports uploading MP4/WebM files with custom controls, autoplay, loop, and aspect ratio locks.
- **YouTube Extension:** Converts video links into responsive, sandboxed iframes.
- **Cloudinary Direct Uploader:** Drag-and-drop image uploading directly inside the editor, outputting auto-optimized WebP URLs.
- **Clean Markdown & HTML Output:** Preserves typography, code blocks, blockquotes, and lists.

Cloudinary Media Lifecycle (Upload, CDN Delivery & Auto-Purge)

To prevent orphaned files from accumulating on Cloudinary, the platform implements strict lifecycle management (`src/lib/cloudinary.ts`):

- **Upload:** `POST /api/upload` checks file types, sets folder namespaces (`clickpoint_innovation/blog` , `clickpoint_innovation/team`), and uploads.
- **Deletion:** When an administrator deletes a blog post, case study, or team member image, the server action extracts the Cloudinary `public_id` and invokes `deleteCloudinaryMedia(url)` , immediately invalidating the edge CDN cache.

Real-Time Notification System (SSE + BroadcastChannel)

The admin dashboard receives instant push alerts when new inquiries, reviews, or job applications are submitted without page refresh:

- **Same-Browser Multi-Tab Synchronization:** Uses `BroadcastChannel("clickpoint_realtime_notifications_v1")` to broadcast events across open tabs without hitting the server.
- **Cross-Browser / Multi-Device Server Synchronization:** Uses Server-Sent Events via `GET /api/notifications/stream` (`src/lib/sse-bus.ts`). New events are pushed to all connected admin clients via persistent HTTP streaming.
- **Audio Alerts:** Configured in `src/components/admin/dashboard/` to play a subtle chime when high-priority leads arrive.

Automated Email Dispatch & HR Candidate Tracking System (ATS)

The dashboard includes a dedicated **Email Composer Modal** (`src/components/admin/email-composer-modal.tsx`) and status manager for job applicants:

- **Status Updates:** Moving an applicant from `PENDING` to `REVIEWING` , `SHORTLISTED` , `REJECTED` , or `HIRED` triggers `POST /api/jobs/send-email` .
- **Templating:** Uses responsive HTML templates (`src/lib/email-templates/`) containing official Clickpoint branding, candidate personalization, role specifics, next steps, and recruiter signatures.

- **Delivery:** Dispatched via Nodemailer with inline CID logo embedding to guarantee 100% rendering fidelity across Gmail, Apple Mail, and Microsoft Outlook.

---

6. AI Chatbot Subsystem (RAG + pgvector + Gemini)

Dual-Mode Architecture

```
[User Chat Input]
        |
        ├── [Tier 1] Static Intent Check (<10ms match for greetings, no emojis)
        |
        ├── [Tier 2] Docker / VPS Microservice Stack
        |     ├── FastAPI Chat Service (:8002)
        |     ├── Redis Response Cache (:6379)
        |     ├── pgvector Semantic Candidate Search (:5432)
        |     └── Google Gemini REST Synthesis
        |
        ├── [Tier 3] Serverless Vercel Supabase Direct RAG
        |     ├── Prisma Direct Extraction across 13 DB Tables
        |     └── Google Gemini Multi-Model Fallback Chain
        |
        └── [Tier 4] Offline Safety Fallback
```

Automatic Admin Mutation Reindexing Pipeline

Whenever any content is saved in the admin CMS, `triggerReindexBackground()` executes:

```
POST http://localhost:8001/reindex
Header: X-Internal-Key: <INTERNAL_API_KEY>
```

The ingestion service re-reads the 13 Prisma tables, generates 384-dimensional embeddings using `sentence-transformers/all-MiniLM-L6-v2`, repopulates `knowledge_chunks`, and flushes Redis caches.

Domain Candidate Injection & Multi-Domain Re-Ranking

Dense vector semantic similarity can experience cosine drift for specialized catalog queries. The retrieval engine (`apps/ai/chat-service/app/retrieval/retriever.py`) resolves this using:

1. **Domain Candidate Injection:** Injects domain-specific candidates for careers, leadership, blogs, and case studies even if initial cosine similarity was low.
2. **Re-Ranking Formulas:**
   - **Careers:** +0.60 boost for active vacancy chunks, +0.25 boost for role titles (`developer`, `marketing`, `intern`).
   - **Leadership:** +0.65 boost for exact leadership bios (Founder, CEO, CTO).
   - **Blogs & Case Studies:** +0.50 boost for published articles and case studies.
3. **Explicit Listing Intent:** Recognizes queries like *"What jobs or internships are open?"* and retrieves the complete set of active positions.

Strict Typography & Zero-Emoji Directives

- All bot greetings and responses are strictly zero-emoji (no 👋, 🚀, etc.).
- Outputs are sanitized using regex (`clean_output_markdown` in Python, `sanitizeReply` in Next.js) to strip rogue unicode emojis and ensure clean single-slash markdown links (`[Careers](/careers)`).

---

7. Security, Authentication & RBAC Architecture

Better Auth Engine & Session Lifecycle

Administrative authentication is managed by **Better Auth** with the Prisma adapter:

- Passwords hashed using salted algorithms.
- Sessions maintained via signed HTTP-only cookies (`better-auth.session_token`).
- Automatic session rotation and expiration enforcement.

Next.js Route Protection (`middleware.ts`)

Every incoming request matching `/admin` or `/admin/:path*` is evaluated before rendering:

```
export async function middleware(req: NextRequest) {
  const sessionToken = getSessionCookie(req);
  const path = req.nextUrl.pathname;

  // Unauthenticated user attempting to access dashboard -> redirect to /admin login
  if (!sessionToken && path.startsWith("/admin/dashboard")) {
    return NextResponse.redirect(new URL("/admin", req.url));
  }

  // Authenticated user visiting login route -> redirect to dashboard
  if (sessionToken && (path === "/admin" || path === "/admin/login")) {
    return NextResponse.redirect(new URL("/admin/dashboard", req.url));
  }

  return NextResponse.next();
}
```

Brute-Force Rate Limiting & Account Security

- Configured in `src/lib/auth.ts`:
  - Global window: 4 requests per 60 seconds.
  - Dedicated `/sign-in/email` rule: **Maximum 4 attempts per 15-minute window**.
  - Prevents credential stuffing and automated dictionary attacks.

Service-to-Service Security Header (`X-Internal-Key`)

Internal communication between the Next.js web application and the Python AI microservices requires the `X-Internal-Key` header matching `INTERNAL_API_KEY`. Requests lacking this header are rejected with `HTTP 401 Unauthorized`.

8. API Route Specifications & Gateway Reference

HTTP Method	Route Endpoint	Access Level	Description & Functional Purpose
`POST`	`/api/chat`	Public	Primary chatbot interaction endpoint for client widget.
`POST`	`/api/chatbot/reindex`	Admin / Internal	Triggers complete knowledge base reindexing.
`GET` / `PUT`	`/api/chatbot`	Admin	Fetches or updates chatbot general settings, pricing, and nodes.
`GET` / `PUT`	`/api/landing`	Admin	Reads or updates the 12 sections on the landing page.
`GET` / `PUT`	`/api/about`	Admin	Reads or updates the About Us page content.
`GET` / `PUT`	`/api/services`	Admin	Reads or updates services catalog content.
`GET` / `PUT`	`/api/industries`	Admin	Reads or updates industry sector solutions.
`GET` / `PUT`	`/api/case-studies`	Admin	Reads or updates case study portfolio projects.
`GET` / `PUT`	`/api/journey`	Admin	Reads or updates company timeline eras.
`GET` / `POST`	`/api/blog/posts`	Admin / Public	Lists all articles or creates a new article with rich text.
`GET` / `POST`	`/api/blog/categories`	Admin	Manages blog topic taxonomy categories.
`POST`	`/api/careers/vacancies`	Admin	Creates a new job vacancy.
`PUT` / `DELETE`	`/api/careers/vacancies`	Admin	Edits or deletes an existing job vacancy.
`POST`	`/api/jobs/apply`	Public	Submits a job application with resume file upload.
`GET`	`/api/jobs/applications`	Admin	Lists candidate applications with filter by vacancy.
`POST`	`/api/jobs/send-email`	Admin	Sends branded candidate status emails via Nodemailer.
`GET` / `POST`	`/api/testimonials`	Public / Admin	Submits review (public) or manages moderation queue (admin).
`GET` / `POST`	`/api/faqs`	Public / Admin	Fetches FAQ accordions or mutates Q&A items.

HTTP Method	Route Endpoint	Access Level	Description & Functional Purpose
`POST`	`/api/contact`	Public	Submits a lead inquiry from the contact page.
`POST`	`/api/upload`	Admin	Uploads image or video assets to Cloudinary.
`GET`	`/api/notifications/stream`	Admin	Persistent Server-Sent Events stream for real-time dashboard alerts.
`ALL`	`/api/auth/[...all]`	Public / Auth	Better Auth endpoint handling login, logout, and session checks.

9. DevOps, Docker Stack & Production Runbook

Environment Variables Matrix

Variable	Environment	Required In	Description
`DATABASE_URL`	Supabase Transaction Pooler	Web, Microservices	`postgresql://postgres.[id]:[pass]@aws-0-ap-southeast-1.pooler.supabase.com:6543/postgres?pgbouncer=true`
`DIRECT_URL`	Supabase Direct Connection	Prisma, Migrations	`postgresql://postgres:[pass]@db.[id].supabase.co:5432/postgres`
`REDIS_URL`	Redis Cache Host	Chat Service, Docker	`redis://clickpoint-redis:6379/0` (or `redis://localhost:6380/0`)
`BETTER_AUTH_SECRET`	Secret Key	Next.js Web	32+ character random secret for signing session cookies
`BETTER_AUTH_URL`	Application URL	Next.js Web	Production domain (e.g. `https://clickpoint.com.np`)
`CLOUDINARY_CLOUD_NAME`	Cloudinary Storage	Next.js Web	Cloudinary cloud identifier
`CLOUDINARY_API_KEY`	Cloudinary Storage	Next.js Web	Cloudinary public API key
`CLOUDINARY_API_SECRET`	Cloudinary Storage	Next.js Web	Cloudinary private secret
`GEMINI_API_KEY`	Google AI Studio	Next.js, Microservices	Google Gemini LLM API key
`GEMINI_MODEL_NAME`	Model Identifier	Next.js, Microservices	Default: `gemini-flash-lite-latest`
`INTERNAL_API_KEY`	Shared Internal Key	All Services	Service-to-service authentication header
`SMTP_HOST`	Email Dispatch	Next.js Web	SMTP server host (e.g. `smtp.gmail.com`)
`SMTP_PORT`	Email Dispatch	Next.js Web	SMTP port (`587` for TLS, `465` for SSL)
`SMTP_USER`	Email Dispatch	Next.js Web	SMTP sender account email address
`SMTP_PASS`	Email Dispatch	Next.js Web	SMTP app password

Local Development Commands

```
# 1. Install dependencies
pnpm install

# 2. Run database migrations / push schema
npx prisma db push

# 3. Seed database with initial default content
pnpm db:seed

# 4. Start Next.js development server
pnpm dev

# 5. Or run all services concurrently (Web + Chat + Ingestion)
pnpm dev:all
```

Docker Compose Multi-Container Deployment

To deploy the entire stack on a dedicated Linux VPS or local Docker host:

```
# Start production containers in background
docker compose -f docker-compose.prod.yml up -d --build

# Verify container health
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Managed Containers:

- `clickpoint-postgres` (Port `5433:5432` — pgvector database)
- `clickpoint-redis` (Port `6380:6379` — caching layer)
- `clickpoint-ingestion-service` (Port `8001:8001` — embedding generator)
- `clickpoint-chat-service` (Port `8002:8002` — vector search & RAG API)
- `clickpoint-web` (Port `3001:3000` — Next.js production web app)

Vercel Production Deployment Workflow

When pushing to the `main` branch:

1. GitHub triggers a production deployment to Vercel.
2. Vercel executes `pnpm build` (`next build`), compiling server components and route handlers.
3. In serverless runtime, `src/app/api/chat/route.ts` detects `process.env.VERCEL = "1"` :
   - Bypasses unreachable localhost microservice ports.
   - Executes direct Prisma batch extraction from Supabase PostgreSQL across all 13 tables.
   - Synthesizes answers using Google Gemini REST APIs with automatic model fallback (`gemini-flash-lite-latest` -> `gemini-3.5-flash`).

Database Migration & Seeding Procedures

```
# Push schema updates to remote Supabase database
npx prisma db push

# Generate updated Prisma Client types
npx prisma generate

# Execute seeding script to populate initial page singletons and admin user
npx tsx prisma/seed.ts
```

Conclusion & Summary

The Clickpoint Innovation platform combines modern React Server Components, type-safe Prisma data modeling, self-hosted Better Auth authentication, real-time administrative workflows (SSE & Cloudinary), and dual-mode conversational AI.

For architectural questions or feature additions, reference this document or consult the Core Engineering Team at `dev@clickpoint.com.np`.